

Product Requirements Document (PRD)

Product Name: ChatSphere (WhatsApp-like Web Chat Application)

Tech Stack: Next.js (Frontend + Backend), MongoDB

1. Overview

ChatSphere is a real-time web-based chat application similar to WhatsApp, enabling users to send text messages, media files, voice notes, and create group chats. The application will be built entirely using **Next.js** (App Router + API routes) and **MongoDB** for persistent storage.

The goal is to build a scalable, production-ready, real-time messaging system optimized for performance, security, and extensibility.

2. Objectives

- Provide real-time 1-to-1 messaging
 - Support group chats
 - Media sharing (images, videos, files)
 - Online/offline presence
 - Message status (sent, delivered, read)
 - Authentication & user management
 - Clean, responsive UI
-

3. Target Users

- Students
 - Teams & small businesses
 - Online communities
 - General web users
-

4. Core Features (MVP)

4.1 Authentication & User Management

- Email/password authentication
 - JWT-based session management
 - Profile setup (name, avatar, status)
 - Secure password hashing (bcrypt)
-

4.2 Real-Time Chat

1-to-1 Chat

- Send & receive text messages
- Typing indicator
- Read receipts
- Message timestamps

Group Chat

- Create group
 - Add/remove participants
 - Group name & icon
 - Admin role
-

4.3 Media Support

- Upload images, videos, PDFs
- Preview media before sending
- Store files (Cloud storage like AWS S3 or Cloudinary recommended)

- Store file metadata in MongoDB
-

4.4 Presence & Status

- Online/offline detection
 - Last seen
 - User status message
-

4.5 Message States

- Sent
 - Delivered
 - Read
-

4.6 Notifications

- Browser push notifications
 - Sound alerts
-

5. Future Features (Phase 2+)

- Voice messages
 - Voice/video calling (WebRTC)
 - Message reactions
 - Message edit/delete
 - Pinned chats
 - End-to-end encryption
 - Multi-device support
 - AI smart replies
 - Chatbot integration
-

6. 🏗️ Technical Architecture

6.1 Frontend (Next.js)

- Next.js App Router
 - Server Components for layout
 - Client Components for chat UI
 - Zustand or Redux Toolkit for state management
 - TailwindCSS for styling
 - WebSockets (Socket.io) for real-time communication
-

6.2 Backend (Next.js API Routes)

- REST APIs for:
 - Auth
 - User management
 - Chat creation
 - Message retrieval
 - WebSocket server (Socket.io)
 - Middleware for JWT verification
-

6.3 Database (MongoDB)

Collections Design

Users Collection

```
{
  _id,
  name,
  email,
  passwordHash,
  avatar,
  status,
  lastSeen,
  createdAt
}
```

Chats Collection

```
{
  _id,
  isGroup,
  participants: [userId],
  groupName,
  groupAdmin,
  createdAt
}
```

Messages Collection

```
{
  _id,
  chatId,
  senderId,
  content,
  messageType: "text | image | video | file",
  fileUrl,
  readBy: [userId],
  createdAt
}
```

7. Real-Time Communication Flow

1. User sends message
 2. Frontend emits event via WebSocket
 3. Backend:
 - Validates JWT
 - Saves message to MongoDB
 - Broadcasts to participants
 4. Clients update UI instantly
-

8. Security Requirements

- JWT authentication
- HTTPS mandatory

- Password hashing (bcrypt)
 - Rate limiting (API abuse prevention)
 - Input validation (Zod)
 - CORS protection
 - XSS & CSRF protection
 - Secure file upload validation
-

9. Performance Requirements

- Message delivery latency < 500ms
 - Infinite scroll with pagination
 - MongoDB indexing:
 - chatId
 - senderId
 - createdAt
 - Use Redis (optional) for presence caching
 - Lazy load media files
-

10. UI/UX Requirements

- Responsive (mobile-first)
 - WhatsApp-like layout:
 - Left sidebar: chats
 - Right panel: messages
 - Dark/Light mode
 - Smooth scrolling
 - Clean minimal interface
-

11. Non-Functional Requirements

Category	Requirement
Scalability	Support 10,000+ concurrent users

Reliability	99.5% uptime
Maintainability	Modular folder structure
Logging	Centralized logging
Monitoring	Use Vercel Analytics / Prometheus

12. Suggested Folder Structure

```
/app
  /chat
  /api
    /auth
    /messages
    /users
/lib
  db.ts
  auth.ts
  socket.ts
/models
  User.ts
  Chat.ts
  Message.ts
/components
```

13. Development Roadmap

Phase 1 (2–3 Weeks)

- Authentication
- Basic chat UI
- 1-to-1 messaging
- MongoDB schema setup

Phase 2 (2 Weeks)

- Group chats
- Media uploads
- Presence system

Phase 3 (2 Weeks)

- Notifications
 - Performance optimization
 - Security hardening
-

14. Success Metrics

- DAU/MAU ratio
 - Message delivery latency
 - User retention
 - Average session duration
 - Crash rate
-

15. DevOps & Deployment

- Deploy on Vercel
 - MongoDB Atlas
 - CI/CD via GitHub Actions
 - Environment variables management
 - Docker (optional)
-

16. Risks & Mitigation

Risk	Mitigation
Scaling WebSockets	Horizontal scaling + Redis adapter

Database performance	Proper indexing
Security breaches	Regular audits
File storage cost	Use CDN + compression

17. 📌 Final Notes

This PRD outlines an MVP-level WhatsApp-like chat application built fully with **Next.js (fullstack)** and **MongoDB**. The architecture is scalable, modern, and production-ready.

If built properly with clean architecture and indexing, this can scale into:

- SaaS messaging platform
 - Internal team communication tool
 - AI-enhanced chat system
 - WebRTC-based calling app
-

If you want, I can now generate:

- ☒ System Design Diagram
- ☒ Database ER Diagram
- ☒ API Documentation
- ☒ Scalable Production Architecture
- ☒ Monorepo structure
- ☒ Full Technical Architecture Document
- ☒ Implementation plan with sprint tasks

Let's build this properly — not just a chat app, but a real-time system that can scale. 🚀